Programming with Python



9. OOPs

- ✓ Types of Programming
- ✓ Introduction to OOPs concepts
- ✓ Class and Attributes
- ✓ Inheritance
- ✓ Types of Attributes
- ✓ Method Overriding
- ✓ Operator Overloading

The zen of Python – PEP20

```
In [1]: import this
```

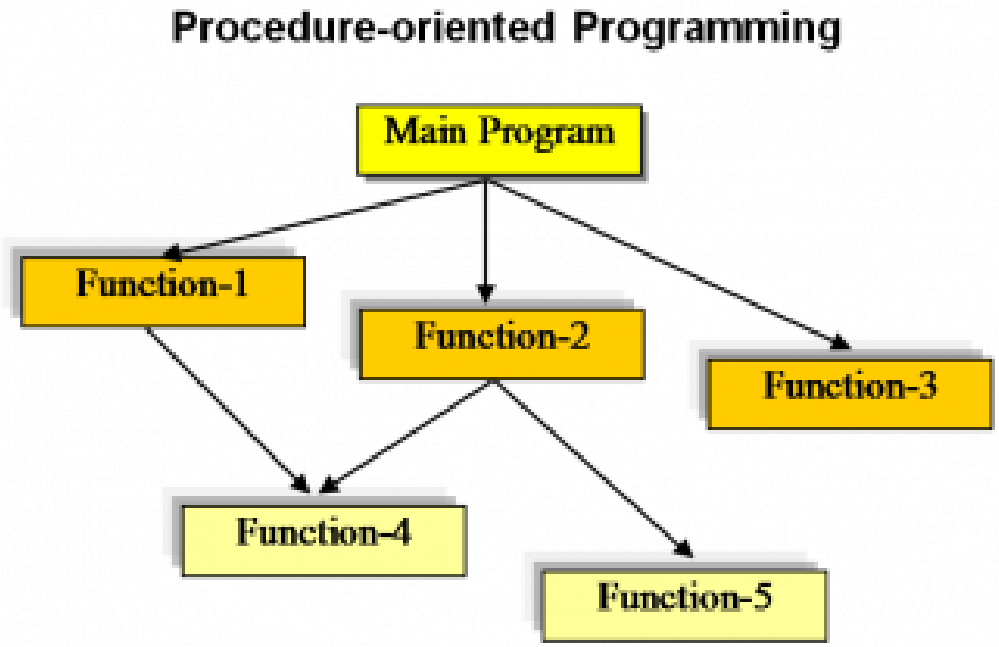
```
The Zen of Python, by Tim Peters

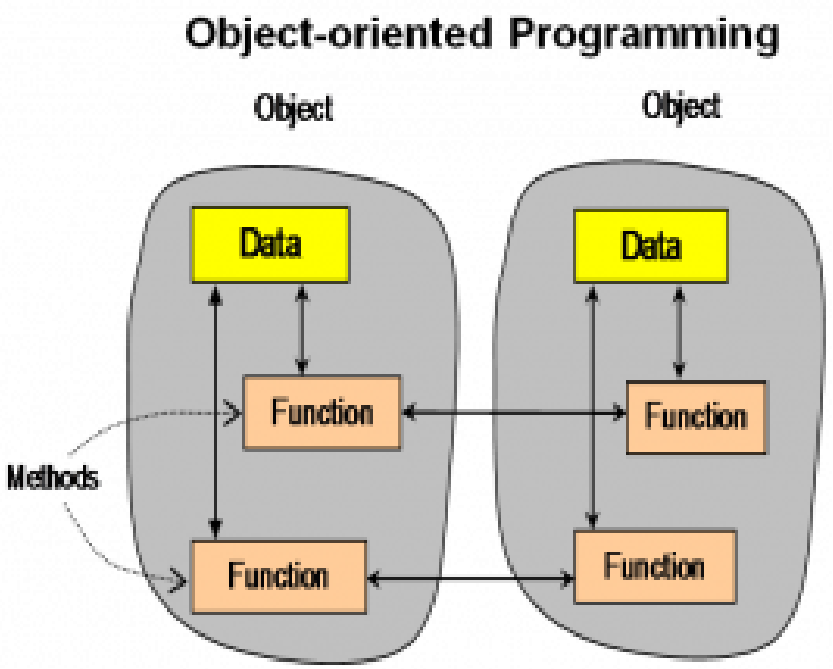
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```



9. Object Oriented Programming (OOP)

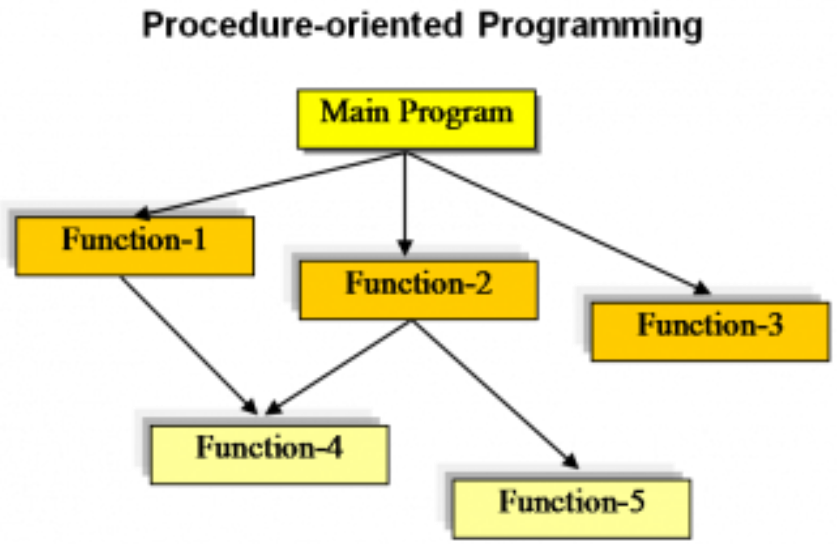
Types of Programming





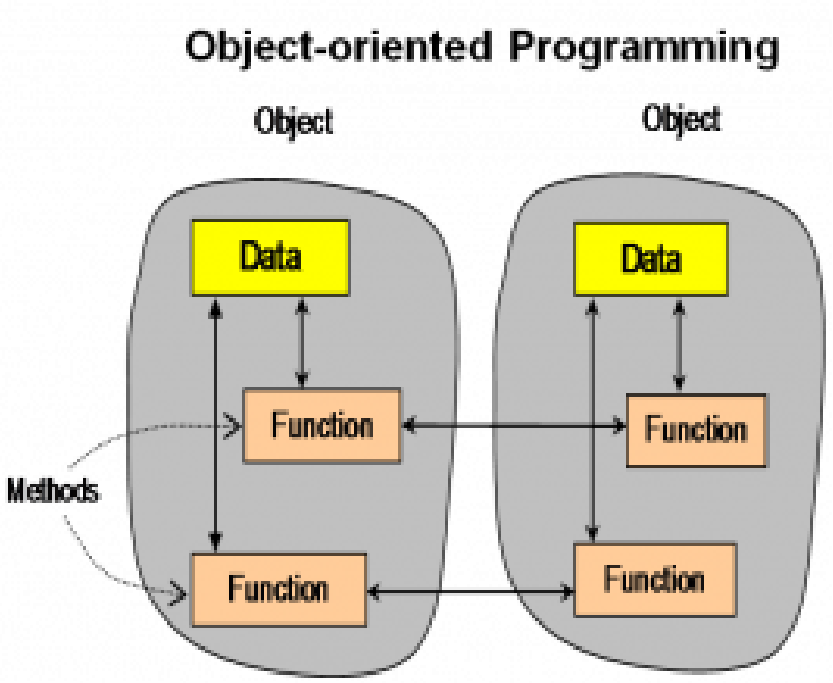
Procedure Oriented Programming

- Define tasks to be performed
- Break tasks into smaller and smaller pieces
  - Until you reach an implementable size
- Define the data to be manipulated
- Design how functions interact
  - What's the input
  - What's the output
- Group functions into components (“modules" or "classes")
- Write the code



Object Oriented Programming

- **First goal:** Model the objects of the world.
  - Noun-oriented
  - Focus on the domain of the program
- **Phases**
  - Object-oriented **analysis**: Understand the domain
    - Define an object-based model of it
  - Object-oriented **design**: Define an implementation
    - Design the solution
  - Object-oriented **programming**: Build it



Objects: Where they came from??

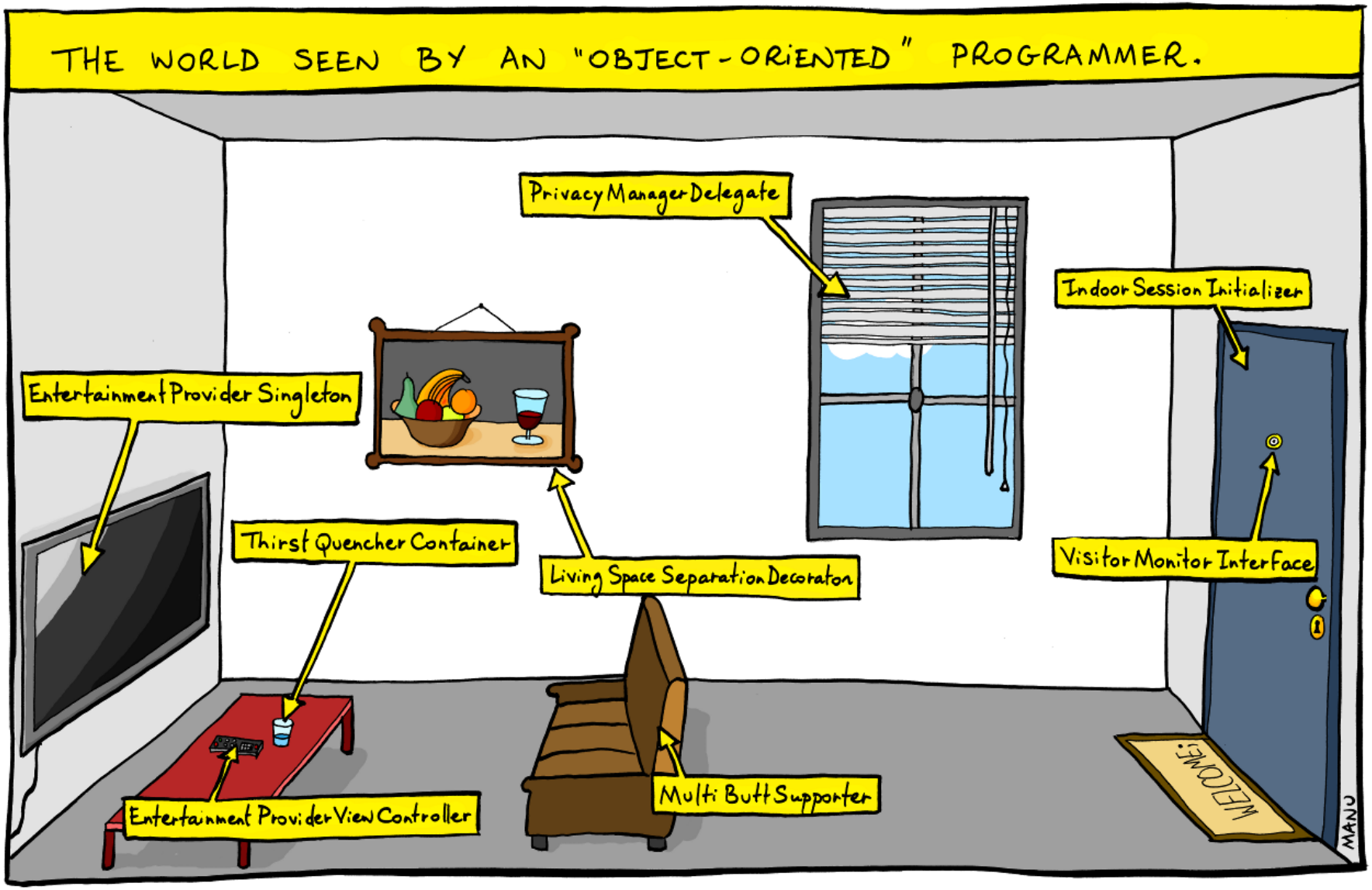


Image Courtesy: http://www.bonkersworld.net/object-world/

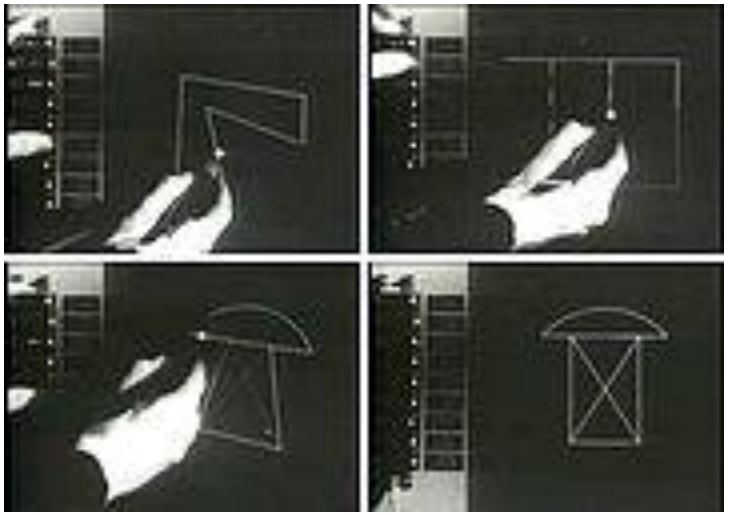
Objects: Where they came from??

1. Sketchpad: Started from 1960s – 70s

- Master-drawings using a software called **"Sketchpad"**
- **First OOP**
- Developed by Ivan Sutherland
- Turing Award in 1988
- Father of computer graphics



- **How it works?**
  - **Master and instance** drawings
    - Draw a house
    - Make an instance
    - Add a chimney to the master
    - Poof! The instance grows a chimney

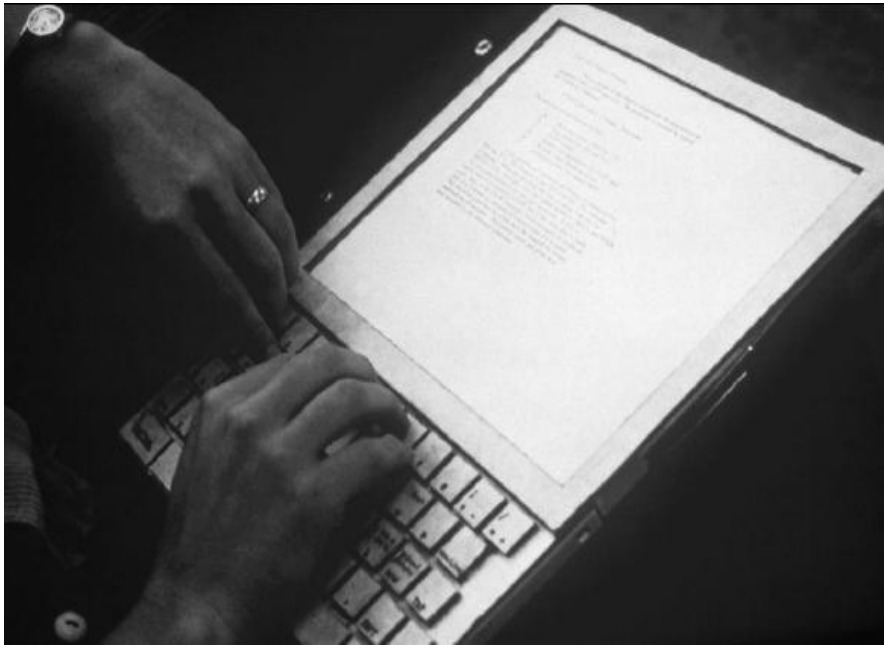


Ref: https://en.wikipedia.org/wiki/Sketchpad

Objects: Where they came from??

2. Simula: Started from 1960s – 70s

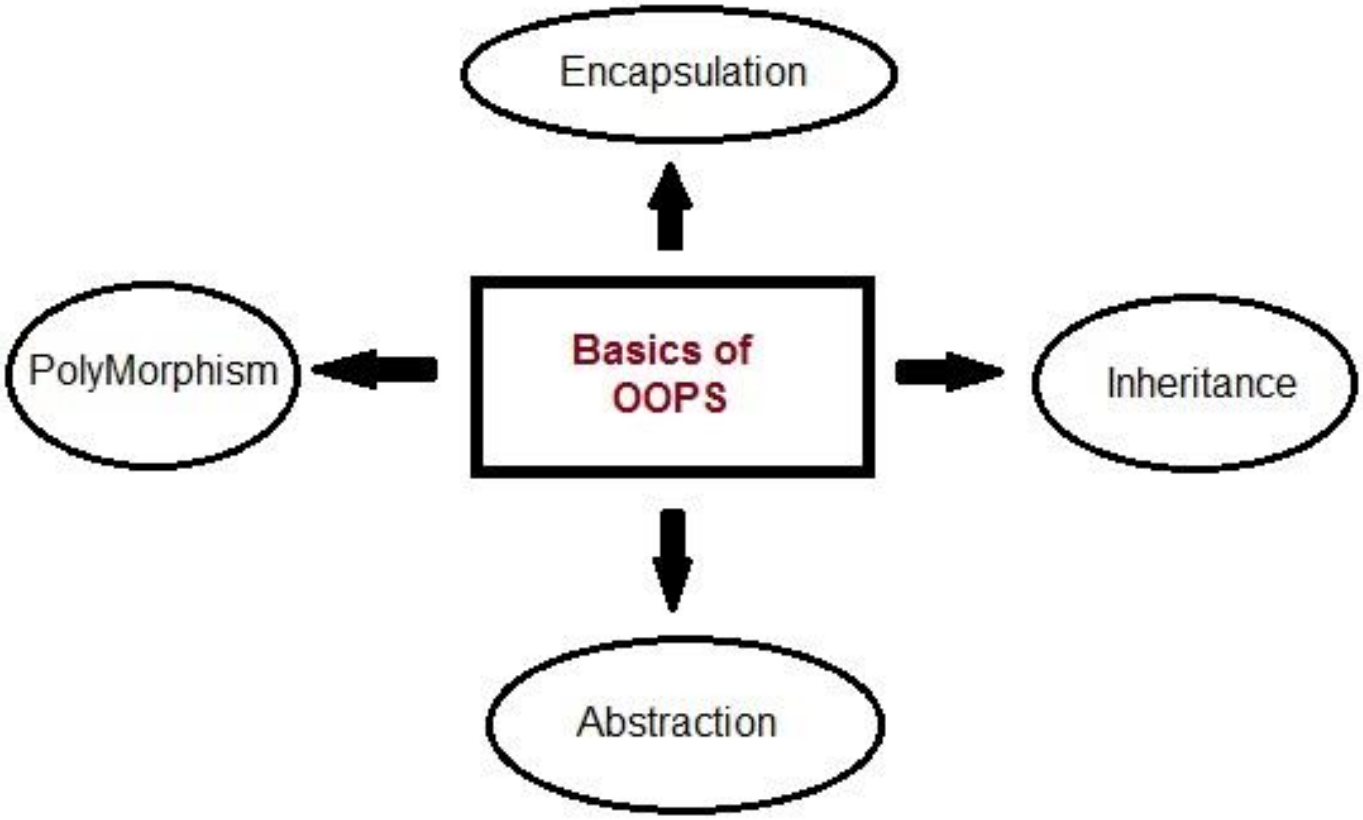
- Simulation programming language from Norway, 1966
- Define an activity which can be instantiated as processes
- Each process has it own data and behavior
  - In real world, objects don't mess with each others' internals directly
- (Simulated) Multi-processing
  - No Universal Scheduler in the Real World



A personal computer for the children of all ages - Dynabook
(OOPs based Machine - Uses Smalltalk)

Ref: https://en.wikipedia.org/wiki/Dynabook

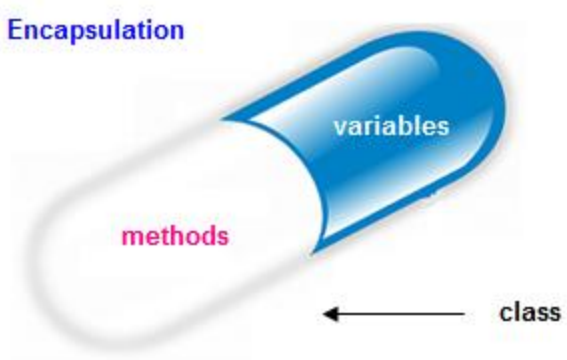
Salient Features of OOP



Encapsulation

“ to enclose in or as in a capsule ”

- The object-oriented meaning of encapsulation is to **enclose related data and methods in a class capsule.**
- **‘bundling together’** of data and behavior.
- **Advantage of Encapsulation (Information hiding) :**
  - It prevents others accessing the insides of an object.
  - The only thing that can manipulate the data in an object is that object’s method or member function.



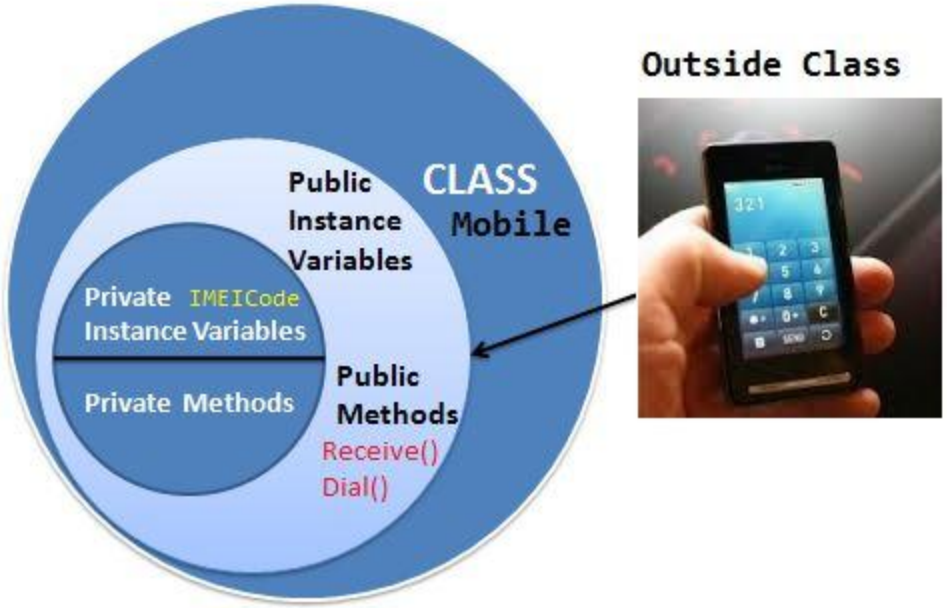
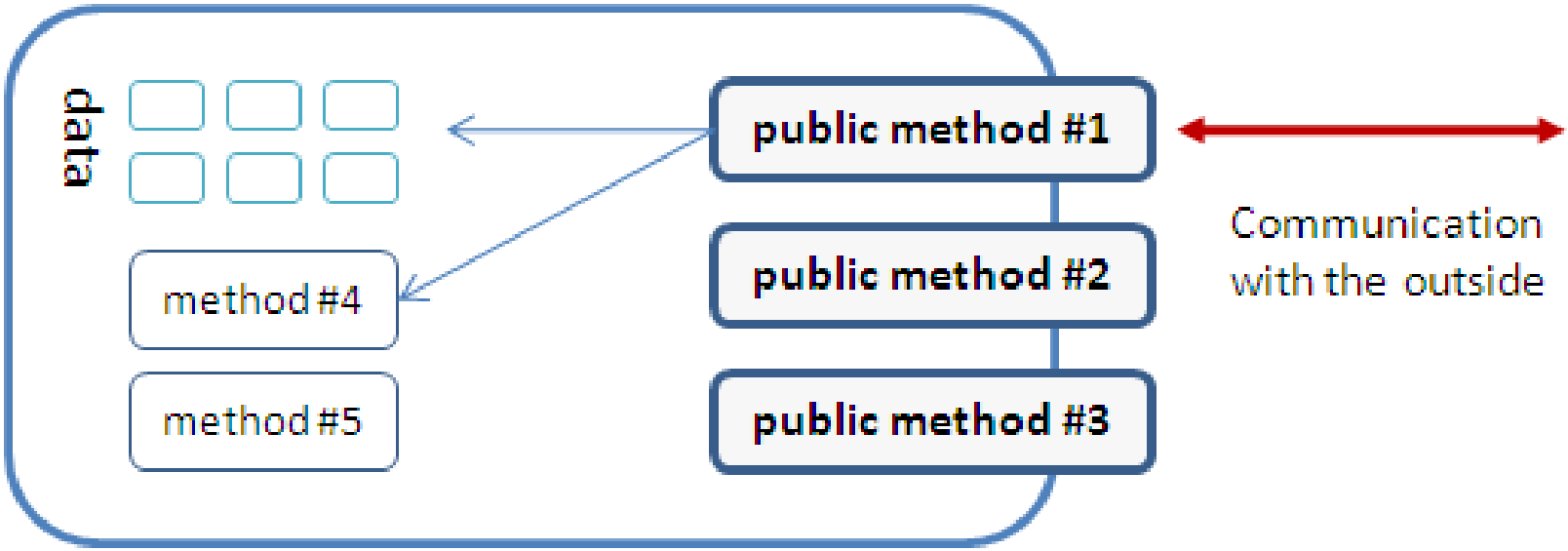


Image Credits: http://www.tutorial4us.com/java/java-encapsulation

Encapsulation…

“ to enclose in or as in a capsule ”



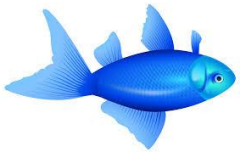
Object and Encapsulation Concept

Image Credits: www.stepwise.hk

Polymorphism

“ A single entity with different forms …”

In object-oriented programming, polymorphism (from the Greek meaning "having multiple forms") is the **characteristic of being able to assign a different meaning** or usage to something in different contexts - specifically, to **allow an entity such as a variable, a function, or an object to have more than one form.** There are several different kinds of polymorphism.

move  → Fish moves by swimming

move  → Birds move by flying

move  → Cats move in their own way

Same function “move”, but the behaviour is different

Inheritance

“ Inherit the properties of the parent (class) and have some extra properties…”

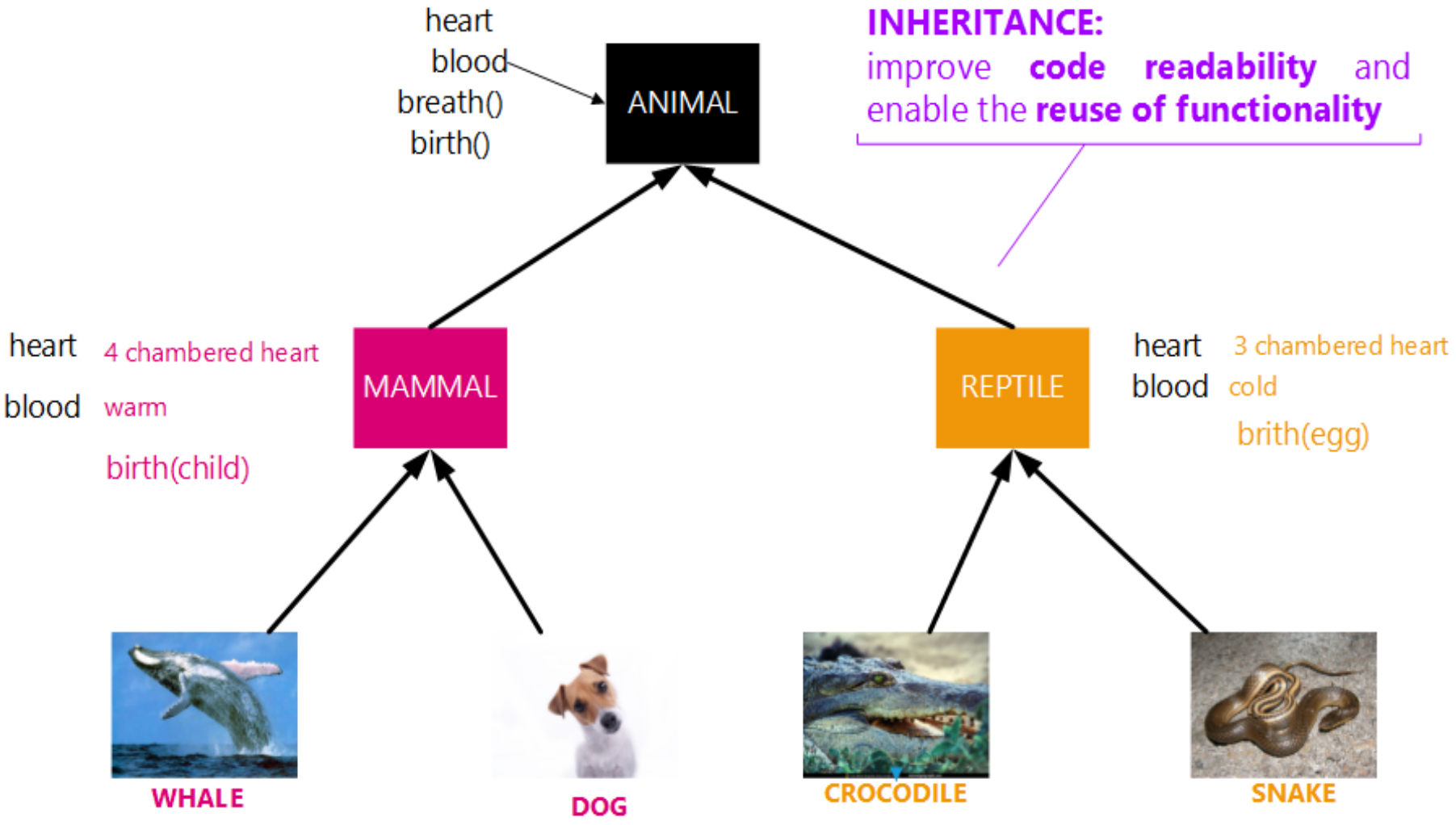


Image Credits: www.stepwise.hk

Advantages of inheritance

- It is an abstraction mechanism which may be used to classify entities.
- It is a reuse mechanism at both the design and the programming level.
- The inheritance graph is a source of organisational knowledge about domains and systems.

Objects and Classes

*An **object** is an entity that has **a state** and a defined **set of operations** which operate on that state. The state is represented as a set of object **attributes.** The operations associated with the object provide services to other objects (clients) which request these services when some computation is required.*

*Objects are created according to some **object class** definition. An object class definition serves as a template for objects. It includes declarations of all the attributes and services which should be associated with an object of that class.*

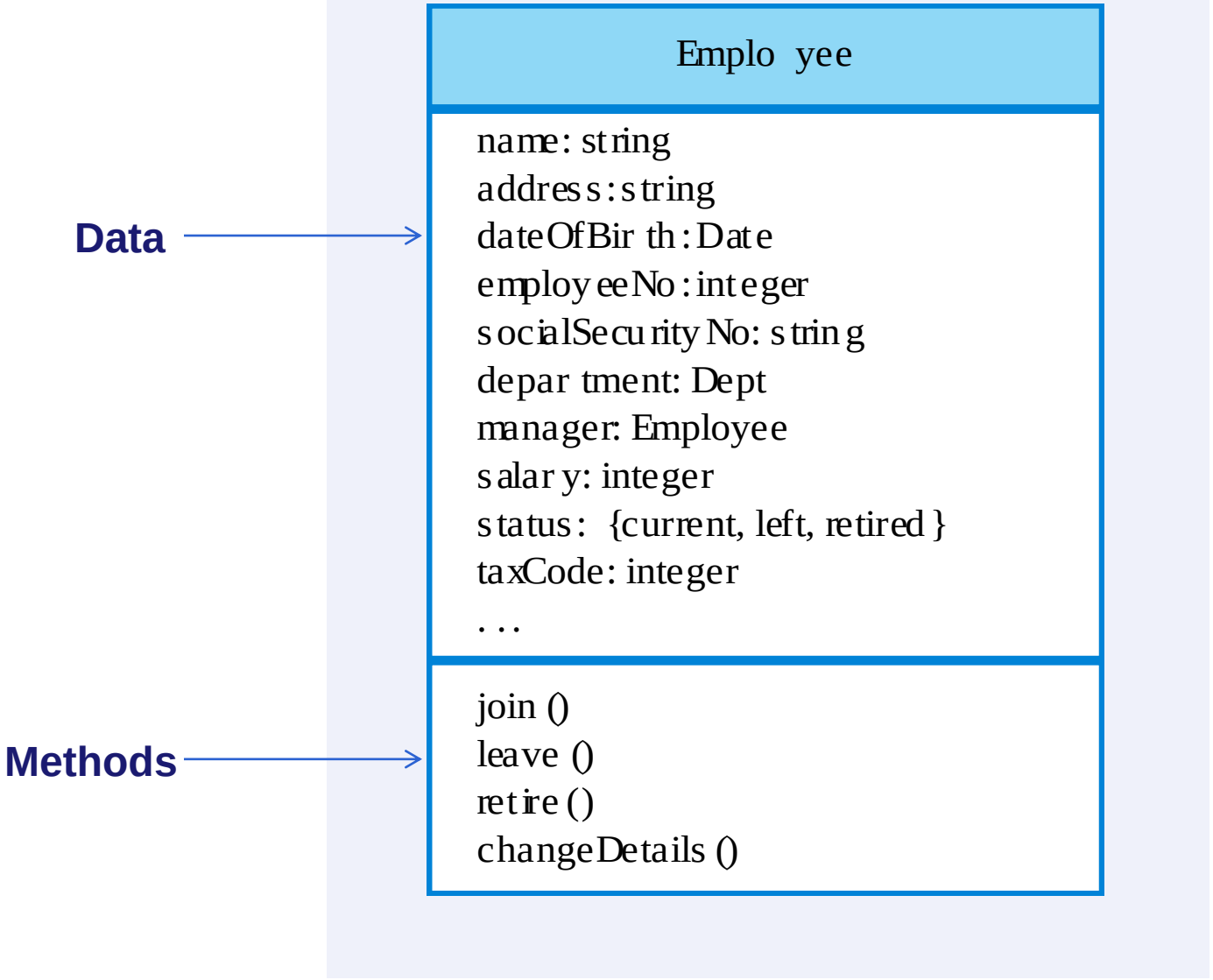
Object communication

- Conceptually, **objects communicate by message passing.**
- **Messages**
  - The name of the service requested by the calling object;
  - Copies of the information required to execute the service and the name of a holder for the result of the service.
- **In practice, messages are often implemented by procedure calls.**
  - Name = procedure name;
  - Information = parameter list.

Advantages of Object Oriented Design

- Easier maintenance.
- Objects may be understood as stand-alone entities.
- Objects are potentially reusable components.
- For some systems, there may be an obvious mapping from real world entities to system objects.

Employee Class



Case Study of OOD

A Weather Mapping System

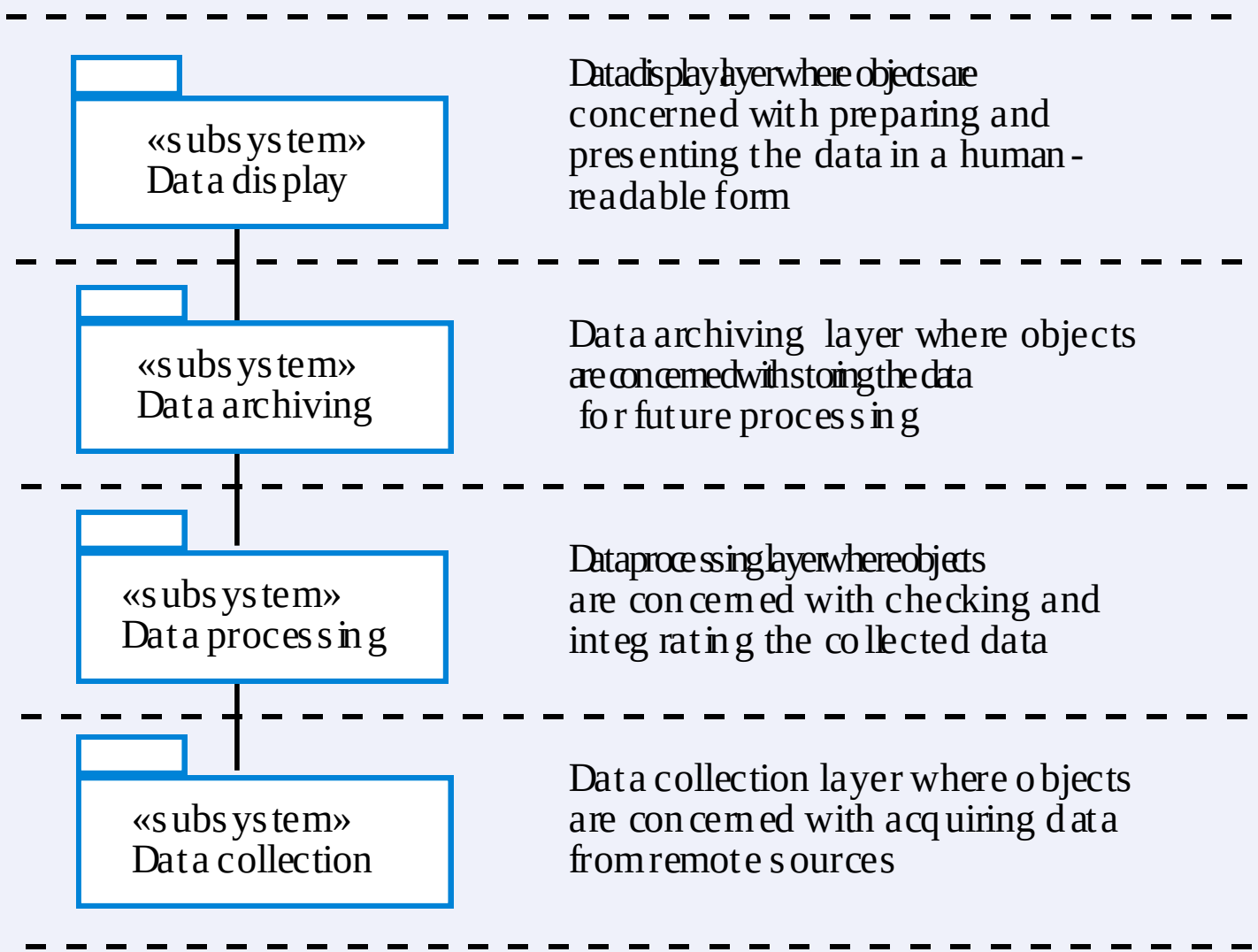
Ref: Software Engineering, By Ian Sommerville

Description

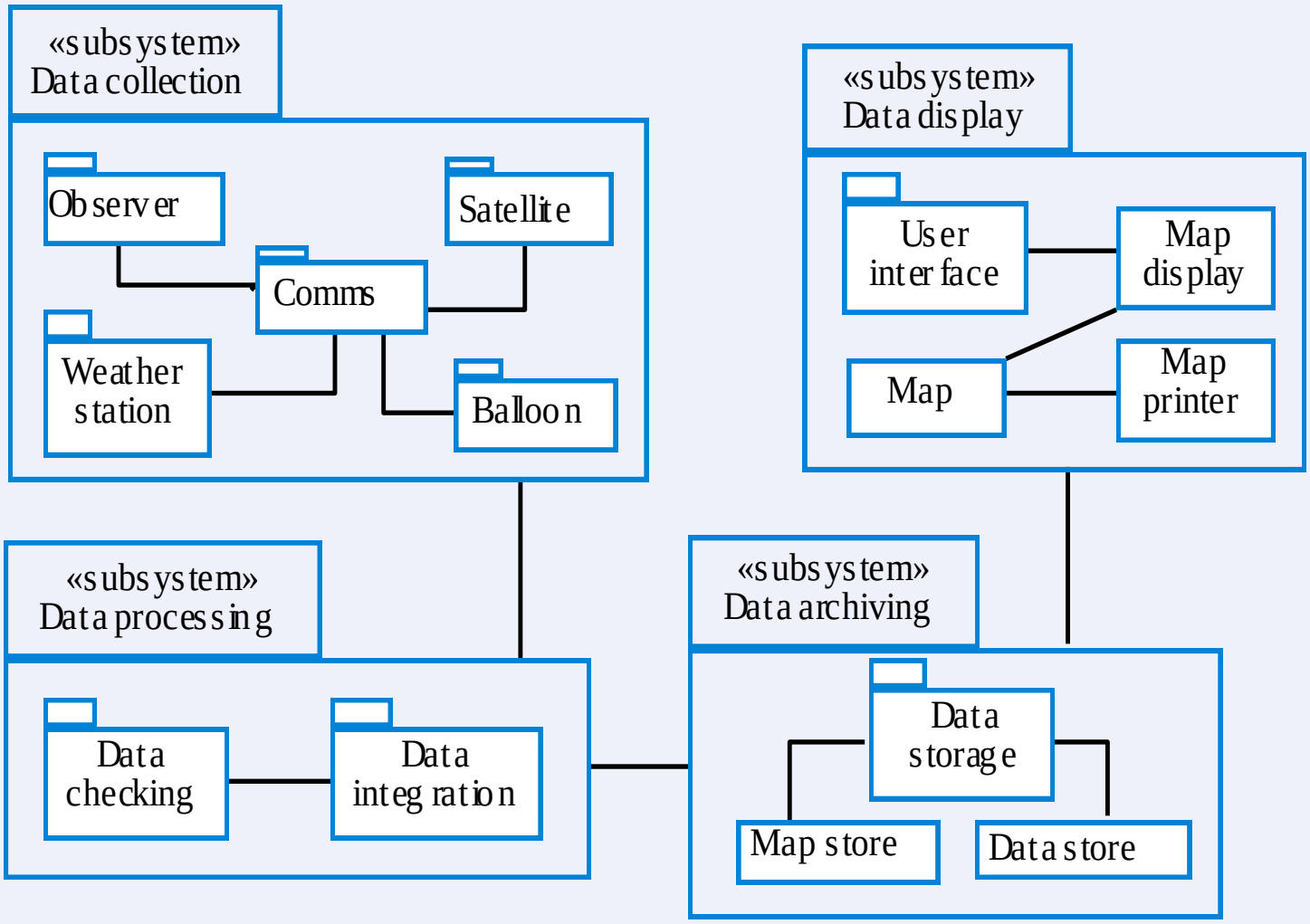
A **weather mapping system** is required to **generate weather maps** on a regular basis **using data collected from remote, unattended weather stations** and other data sources such as weather observers, balloons and satellites. Weather stations transmit their data to the area computer in response to a request from that machine.

The area computer system validates the collected data and **integrates it** with the data from different sources. The **integrated data is archived** and, using data from this archive and a digitised map database **a set of local weather maps are created.** Maps may be printed for distribution on a special-purpose map printer or may be displayed in a number of different formats.

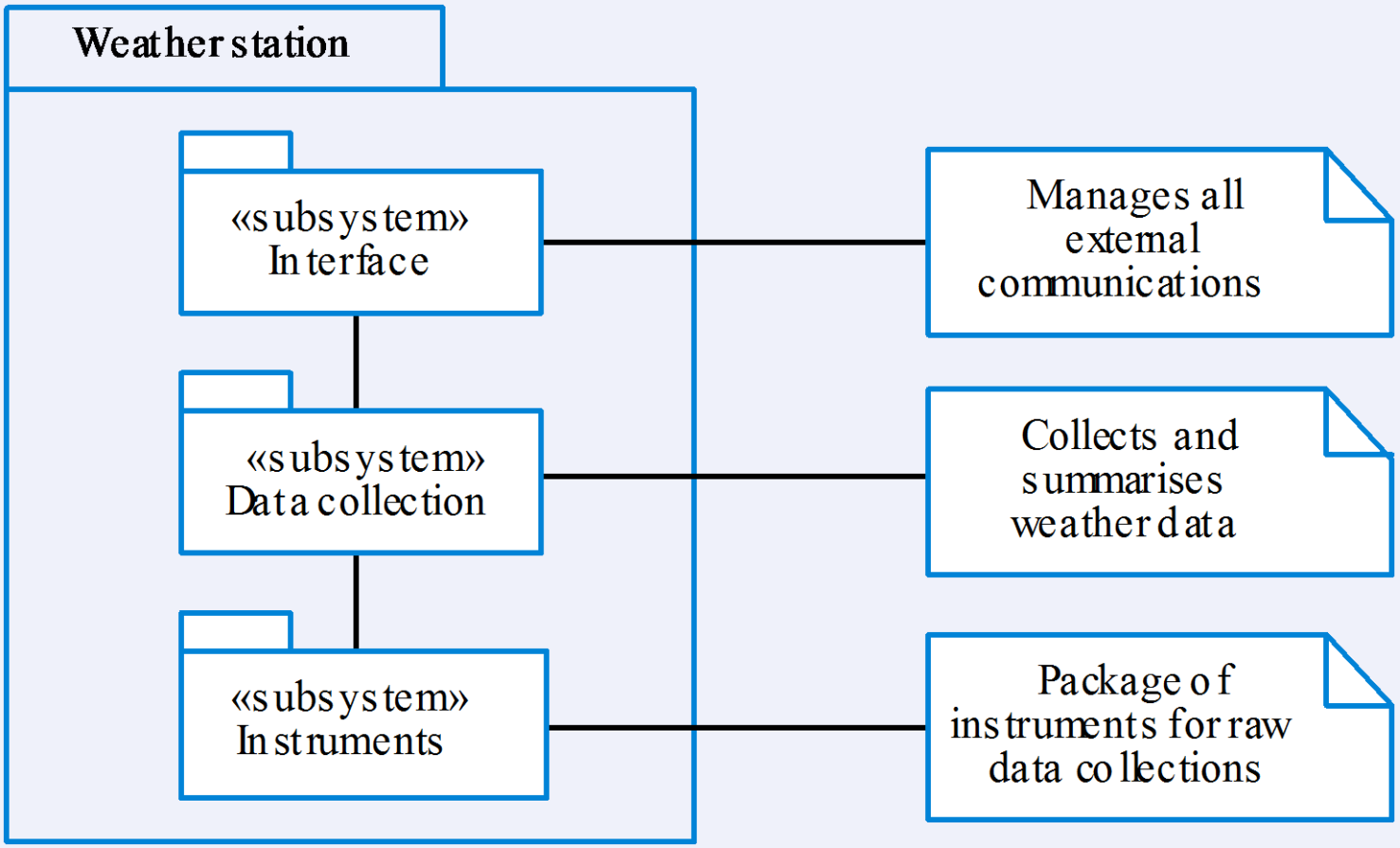
Layered architecture



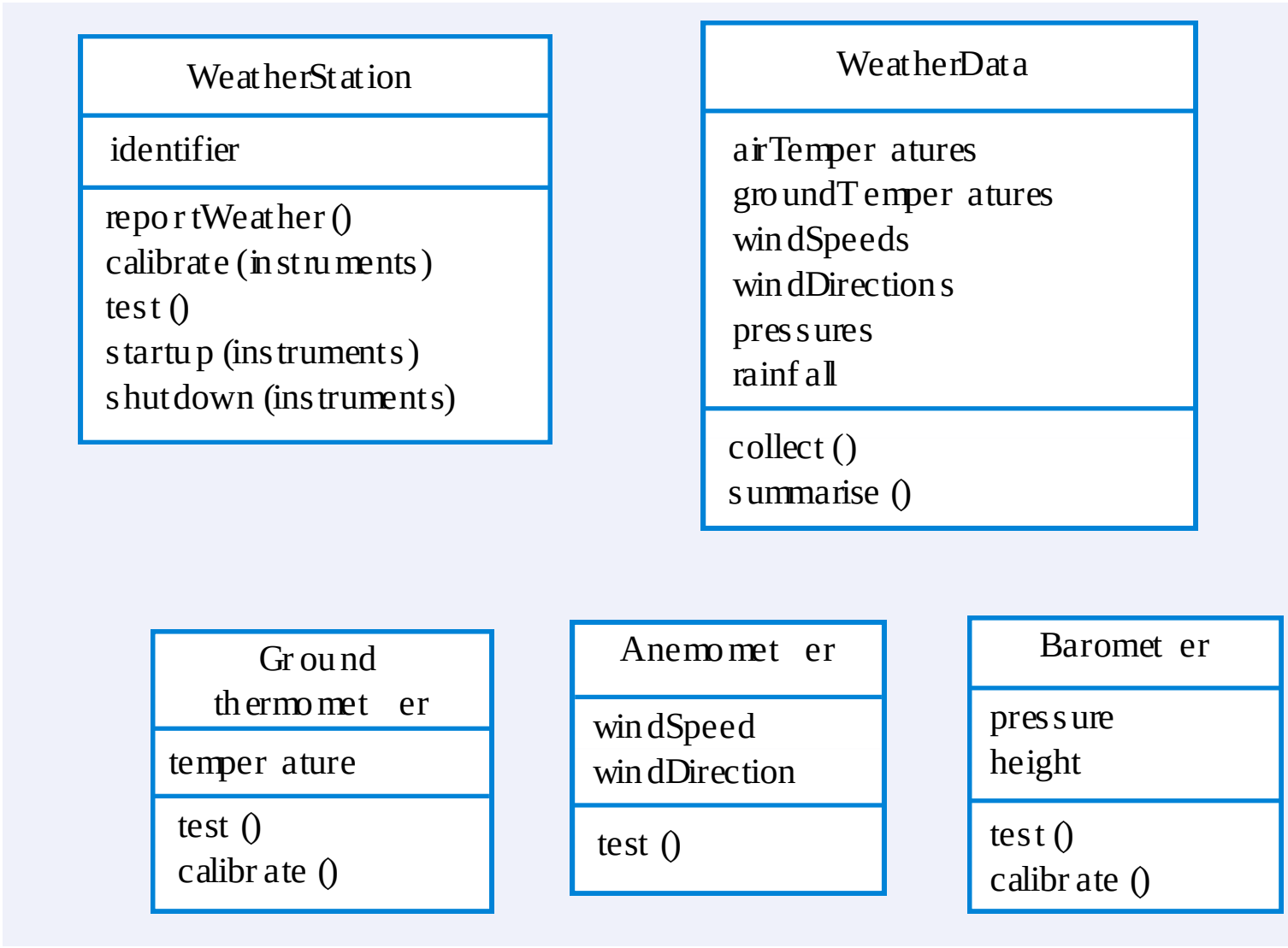
Subsystems in the weather mapping system



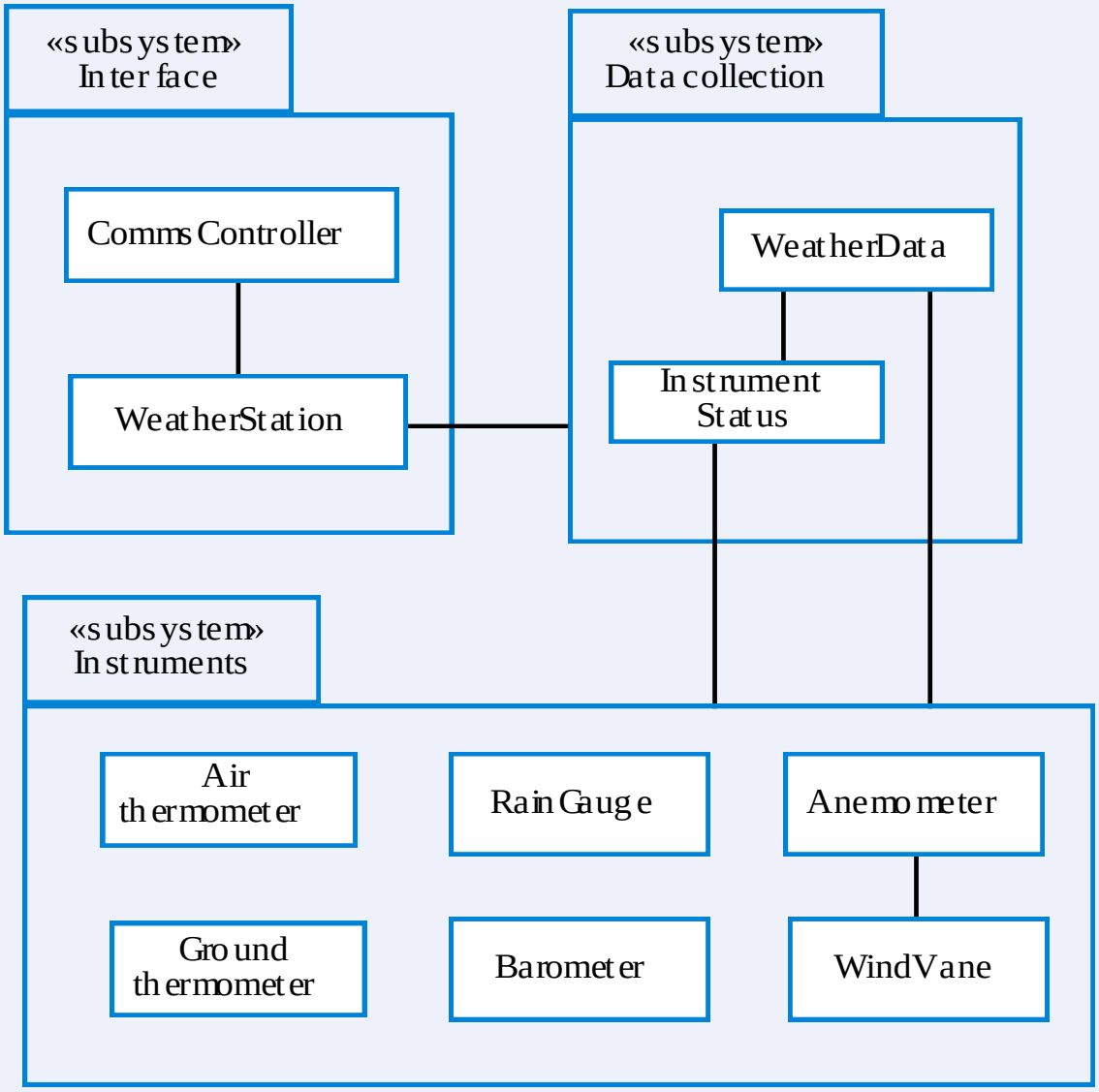
Weather station architecture



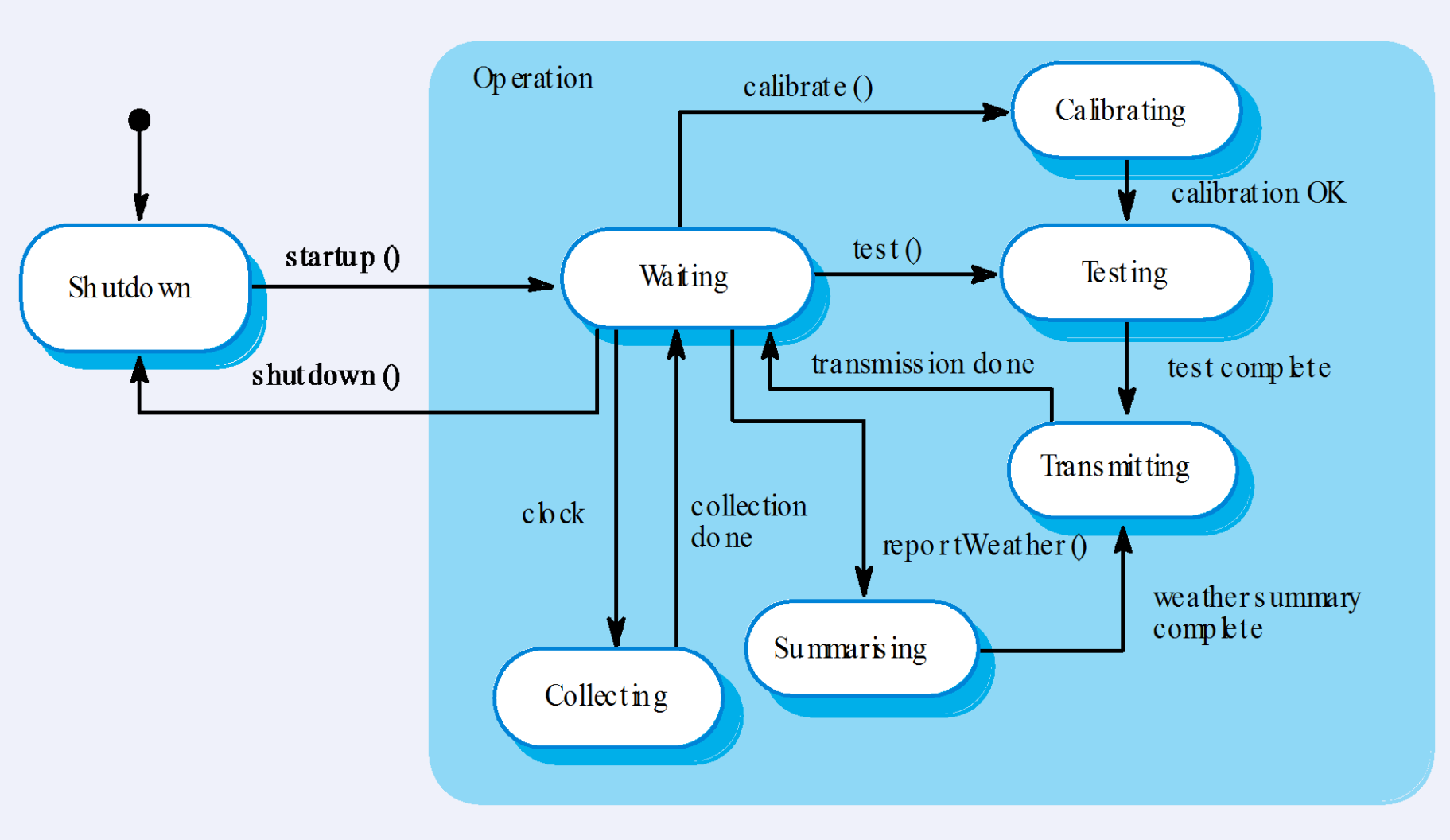
Weather station object classes



Weather station subsystems



Weather station state diagram



Coming back to Python OOP…

Compared with other programing Languages

Few Comparisons/Similarities

- It is a mixture of the class mechanisms found in **C++** and **Modula-3**.
- Python classes provide all the standard features of OOP
- Python **semantics more close to Modula-3**, rather than C++
- Aliasing of names/objects is possible in python, like other languages, But not appreciated in case of mutable objects.

Python Namespaces




```
def printList(upper_limit, step=2):
    # variables upper_limit and step belong to this
     function's local namespace.
    print "upper limit: %d" % upper_limit
    num_list = range(0, upper_limit, step)
    print num_list

printList(upper_limit=5, step=2)
a = "foo"
b = "bar"
# variables a, b and function printList belong to
Global namespace of this module
```

Python Namespaces…

- A *namespace* is a collection of names
- Different namespaces exist at a given time but are isolated.
- After starting the interpreter, all the built-namespaces will be created automatically and exist till the end of the execution.

Scope and Namespace - Example

scopes and namespaces, and how global and nonlocal affect variable binding.

```
def scope_test():
    def do_local():
        spam = "local spam"
    def do_nonlocal():
        nonlocal spam
    def do_global():
        global spam
        spam = "global spam"
    spam = "test spam"
    do_local()
    print("After local assignment:", spam)
    do_nonlocal()
    print("After nonlocal assignment:", spam)
    do_global()
    print("After global assignment:", spam)
scope_test()
print("In global scope:", spam)
```

The nonlocal statement causes the listed identifiers to refer to pre bound variables in the nearest enclosing scope excluding globals.

Output

```
After local assignment: test spam
After nonlocal assignment: nonlocal spam
After global assignment: nonlocal spam
In global scope: global spam
```

Python Scopes

- A ***scope*** is a textual region of a Python program where a namespace is directly accessible.
- "Directly accessible" here means that an unqualified reference to a name attempts to find the name in the namespace.
- At any time during execution, there are **at least three nested scopes** whose namespaces are directly accessible
  - **The innermost scope**, which is searched first, contains the **local names**
  - The **scopes of any enclosing functions**, which are searched starting with the nearest enclosing scope, contains **non-local**, but also **non-global** names
  - The **next-to-last scope** contains the **current module's global names**
  - The **outermost scope** (searched last) is the namespace containing **built-in names**

Class

Everything is a class in Python…

"All objects are "instances of classes."---All are First Class

integers, strings, functions, classes, modules, methods, (and so on) to have **equal status**

functions and methods are values just like lists, integers or floats. Each of these are instances of their corresponding classes.

```
# What is the type of an int?
>>> x = 30
>>> print(type(x))
<class 'int'>
# What is the type of a function f?
>>> def f(x):
        return x+1
>>> print(type(f))
<class 'function'>
```

Define a Class

- We define a class using the keyword **class**.
- **A class creates a new local namespace** where all its attributes are defined.
- Attributes may be **data** or **functions**.

```
# Class Syntax
class ClassName:
      "Optional class documentation string"
      <statement-1>
      . . .
      <statement-N>
```

Define a Class…

- As soon as we define a class, a new class object is created with the same name.
- This class object allows us to access the different attributes as well as to instantiate new objects of that class.

```
# A Simple Class
class myclass:
        pass
```

- The **pass statement is a *null* operation**; nothing happens when it executes.
- The**pass** is also useful in places where your code will eventually go, but has not been written yet

Simple Example

- A class can be defined with only with variables, without methods

```
# A class to represent complex number
class complex:
      "A class to represent a complex number"
       x = 0
       y = 0
```

```
# How to access class members
complex.x = 5.0
Complex.y = 10.0
print(complex.x, complex.y)
Output:
5.0   10.0
```

What is the value of **print(complex.__doc__)**?

Class Methods

- We can define methods in classes.

```
# A class to represent complex number
class complex:
      "A class to represent a complex number"
       x = 0
       y = 0

      # A method to perform print operation
      def printc(self, x,y):
            print(x, y)
      def cmod(self, x,y):
            print(x*x + y*y)
```

```
C1 = complex()
C1.printc(c1.x, c1.y)
C1.cmod(c1.x, c1.y)
```

Class Methods…vs…Functions

- **A method differs from a function only in two aspects:**
  - It belongs to a class and it is defined within a class
  - The first parameter in the definition of a method has to be a reference to the instance, which called the method. This parameter is usually called "self".
  - Most object-oriented programming languages pass the reference to the object (self) as a hidden parameter to the methods.
- **Note:**
- For a **Class C**, an **instance x of C** and **a method m** of C the following three method calls are equivalent:
  - type(x).m(x, ...)
  - C.m(x, ...)
  - x.m(...)

Class Constructor

- Initialize the attributes of an instance right after its creation
- **__init__** is a method which is immediately and automatically called after an instance has been created.
- You can define anywhere in class definition

```
# Complex number class
class complex:
"Constructor"
  def __init__(self, x, y):
     self.real = x
     self.imag = y
  def printc(self)
   print(self.real,slef.imag)
```

```
C1 = complex(1.0, 2.0)
C2 = complex(3.0, 4.0)

C1.printc()
C2.printc()
```

Class Constructor…

What is the output of the following python statements??

print(complex.printc)

<unbound method complex.printc>

print(c.printc)

<bound method complex.printc of <__main__.complex instance at 0xb7e94aac>>

Notice that the self parameter in function definition inside the class. But we called the method simply as **c.printc() without any arguments**. It **still worked.** This is because, whenever an object calls its method, **the object itself is passed as the first argument.** So,

c.printc() → translates into → complex.printc(c).

Delete attributes/Objects

Any attribute of an object can be deleted anytime, using the del statement.

```
# Delete an attribute or Object
C1 = complex(10, 20)
del c1.real
C1.printc()

Output:
AttributeError: 'ComplexNumber' object has no attribute
'getData'

del C1
C1.printc()

Output:
NameError: name 'c1' is not defined
```

Accessing Attributes - Methods

- We can access the object's attributes using the dot operator with object.
- Class variable would be accessed using class name as follows.

  C1.complex( )

  C2.complex( )

You can add, remove, or modify attributes of classes and objects at any time.

- Instead of using the normal statements to access attributes, you can use the following functions:
  - The **getattr(obj, name[, default])** : to access the attribute of object.
  - The **hasattr(obj, name)** : to check if an attribute exists or not.
  - The **setattr(obj, name,value)** : to set an attribute. If attribute does not exist, then it would be created.
  - The **delattr(obj, name)** : to delete an attribute.

Accessing Attributes…

```
# Define an Employee class
class Employee:
    '''Class of employees'''
    empCount = 0

    # Constructor - You can define at any place in the class def.
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.empCount += 1

    def displayCount(self):
        print("Total Number of Employees:", Employee.empCount)

    def displayEmployee(self):
        print("Name : ", self.name, "Salary: ", self.salary)
# Create employee objects and initialize using constructor
emp1 = Employee("Ram", 2000)
emp2 = Employee("Rahim", 5000)
```

Accessing Attributes…

```
# Accessing attributes  using built-in functions
hasattr(emp1, 'age')    # Returns true if 'age' attribute exists
getattr(emp1, 'age')    # Returns value of 'age' attribute
setattr(emp1, 'age', 8) # Set attribute 'age' at 8
delattr(emp1, 'age')    # Delete attribute 'age'
```

Built-In Class Attributes

Every Python class keeps following built-in attributes and they can be accessed using dot operator like any other attribute:

- **__doc__:** Class documentation string or none, if undefined.
- **__name__:** Class name.
- **__module__:** Module name in which the class is defined. This attribute is "__main__" in interactive mode.
- **__bases__:** A possibly empty tuple containing the base classes, in the order of their occurrence in the base class list.

Built-In Class Attributes - Example

```
class Employee:
   'Common base class for all employees'
   empCount = 0
   def __init__(self, name, salary):
      self.name = name
      self.salary = salary
      Employee.empCount += 1

   def displayCount(self):
     print "Total Employee %d" % Employee.empCount

   def displayEmployee(self):
      print "Name : ", self.name,  ", Salary: ", self.salary

print("Employee.__doc__:", Employee.__doc__)
print("Employee.__name__:", Employee.__name__)
print("Employee.__module__:", Employee.__module__)
print("Employee.__bases__:", Employee.__bases__)
print("Employee.__dict__:", Employee.__dict__)
```

Built-In Class Attributes - Example

Output:

```
Employee.__doc__: Common base class for all employees
Employee.__name__: Employee
Employee.__module__: __main__
Employee.__bases__: ()

Employee.__dict__:
{'__module__': '__main__', 'displayCount':
<function displayCount at 0xb7c84994>, 'empCount': 2,
'displayEmployee':    <function    displayEmployee    at
0xb7c8441c>,
'__doc__': 'Common base class for all employees',
'__init__': <function __init__ at 0xb7c846bc>}
```

Fill in the blanks

Fill in the blanks to create a student class

```
class Student _
  def __init__(self, name):
    self.name = name

  ___  sayHi( ___ ):
    print("Hi from "+ ___ .name)

s1 = Student("Amy")
s1.sayHi()
```

Answer: :

def

self

What is the output?

Accessing the attribute which is not defined….

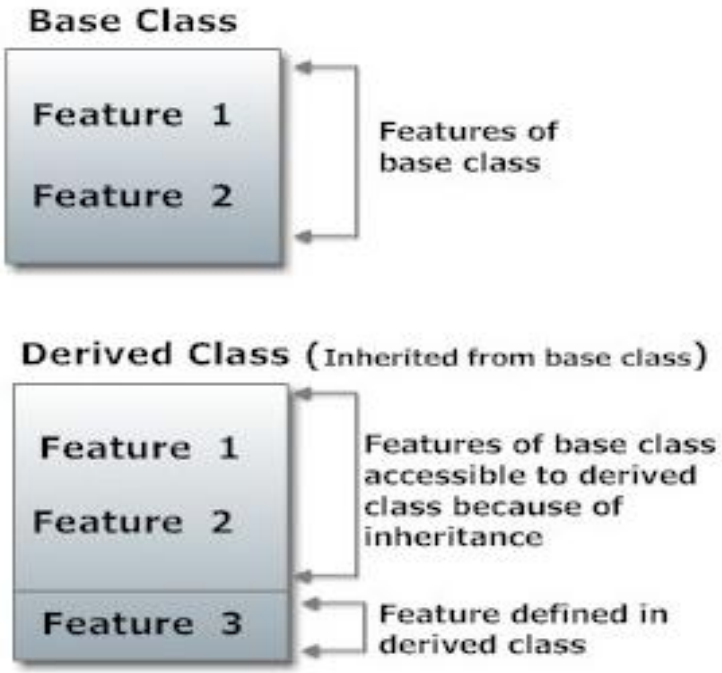
```
class Rectangle:
  def __init__(self, width, height):
    self.width = width
    self.height = height

rect = Rectangle(7, 8)
print(rect.color)
```

```
>>>
AttributeError: 'Rectangle' object has no attribute 'color'
>>>
```

Inheritance

- Allows to define a new class with little or no modification to an existing class.
- The new class is called **derived (or child) class** and the one from which it inherits is called the **base (or parent) class.**
- Derived class inherits features from the base class, adding new features to it.
- This results into re-usability of code.




```
class DerivedClass(BaseClass):
        body_of_derived_class
```

Inheritance - Example

```
# Define a class "Polygon" - Base Class
class Polygon:
    def __init__(self, no_of_sides):
        self.n = no_of_sides
        self.sides = [0 for i in range(no_of_sides)]

    def inputSides(self):
        self.sides = [float(input("Enter side "+str(i+1)+" : ")) for i in range(self.n)]

    def dispSides(self):
        for i in range(self.n):
            print("Side",i+1,"is",self.sides[i])
```

Inheritance – Example…

```
# Define a class "Triangle" - Derived Class
class Triangle(Polygon):
    def __init__(self):
        Polygon.__init__(self,3)

    def findArea(self):
        a, b, c = self.sides
        # calculate the semi-perimeter
        s = (a + b + c) / 2
        area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
        print('The area of the triangle is %0.2f' %area)
```

Inheritance – Example…

```
# Use base class and derived class
>>> t = Triangle()

>>> t.inputSides()
Enter side 1 : 3
Enter side 2 : 5
Enter side 3 : 4

>>> t.dispSides()
Side 1 is 3.0
Side 2 is 5.0
Side 3 is 4.0

>>> t.findArea()
The area of the triangle is 6.00
```

Inheritance – In-Built Functions

- You can use issubclass() or isinstance() functions to check a relationships of two classes and instances.
  - **issubclass(sub, sup):** boolean function returns true if the given subclass **sub** is indeed a subclass of the superclass **sup**.
  - **isinstance(obj, Class):** boolean function returns true if *obj* is an instance of class *Class* or is an instance of a subclass of Class

```
class DerivedClass(BaseClass):
        body_of_derived_class
```

Public, Private and Protected Attributes

Data Hiding

How to protect the data which should be private?

What to do if trespassing, i.e. accessing or changing private data, occurs?

We have a classification in object-oriented programming:

- **Private attributes** should only be used by the owner
- **Protected (restricted) Attributes** may be used, but at your own risk. Essentially, this means that they should only be used under certain conditions.
- **Public Attributes** All member variables and methods are public by default in Python.

Public, Private and Protected Attributes…

Naming Scheme to control the accessibility

Naming	Type	Meaning
name	Public	These attributes can be freely used inside or outside of a class definition.
_name	Protected	Protected attributes should not be used outside of the class definition, unless inside of a subclass definition. Used by base class and sub class only.
__name	Private	This kind of attribute is inaccessible and invisible. It's neither possible to read nor write to those attributes, except inside of the class definition itself.

Public, Private and Protected Attributes…

Example

```
class A():
        def __init__(self):
                self.__priv = "I am private"
                self._prot = "I am protected"
                self.pub = "I am public"
```

```
>>> from attribute_tests import A
>>> x = A()
>>> x.pub 'I am public'
>>> x.pub = x.pub + " and my value can be changed"
>>> x.pub 'I am public and my value can be changed'
>>> x.prot
Traceback (most recent call last): File "", line 1, in AttributeError:
'A' object has no attribute 'prot'
>>> x._prot 'I am protected'
>>> x.__priv Traceback (most recent call last): File "", line 1, in
AttributeError: 'A' object has no attribute '__priv'
>>>
```

Method Overriding

- **We can override the parent class methods.**
- One reason for overriding parent's methods is because you may want special or different functionality in your subclass.

```
class Parent:          # define parent class
    def myMethod(self):
        print('Calling parent method')

class Child(Parent): # define child class
    def myMethod(self):
        print('Calling child method')

c = Child()            # instance of child
c.myMethod()           # child calls overridden method

Output: Calling child method
```

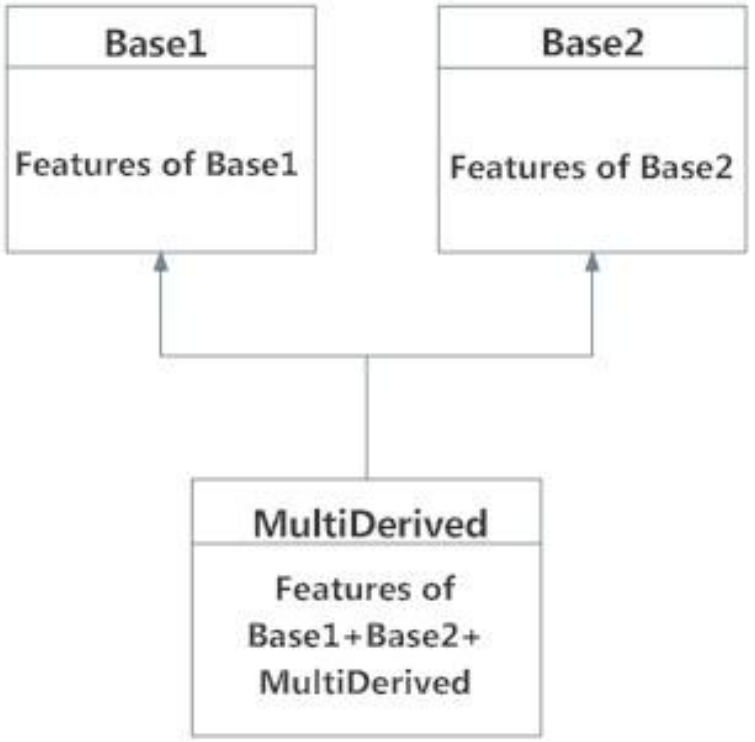
Multiple Inheritance

- A class can be derived from more than one base classes.
- The syntax for multiple inheritance is similar to single inheritance.
- **Syntax:**

```
class Base1:
      pass

class Base2:
      pass

class MultiDer(Base1, Base2):
      pass
```



Multiple Inheritance - Example

```
class A:
    def who_am_i(self):
        print("I am a A")

class B(A):
    def who_am_i(self):
        print("I am a B")

class C(A):
    def who_am_i(self):
        print("I am a C")

class D(B,C):
    def who_am_i(self):
        print("I am a D")

d1 = D()
d1.who_am_i()
```

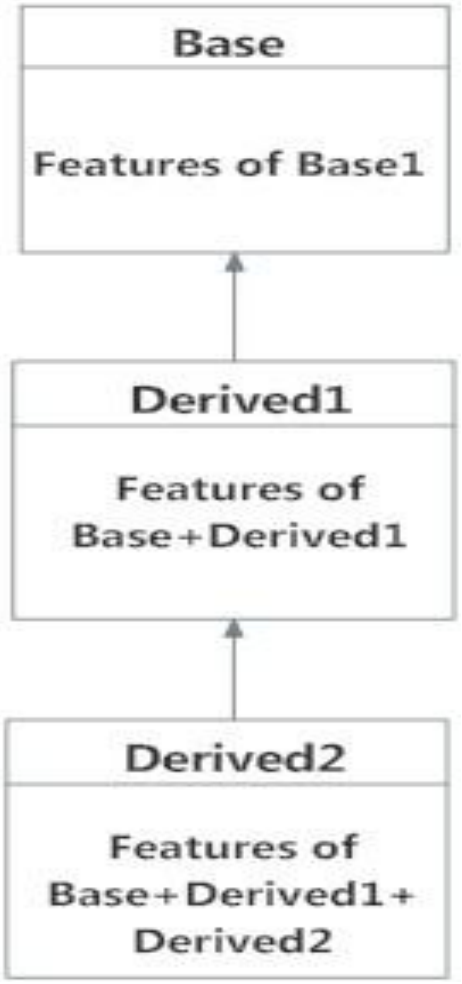
Multilevel Inheritance

- we can inherit form a derived class. This is also called multilevel inheritance. Multilevel inheritance can be of any depth in Python.
- **Syntax:**

```
class Base:
    pass

class Derived1(Base):
    pass

class Derived2(Derived1):
    pass
```



Operator Overloading

- Python operators work for built-in classes.
- But same operator behaves differently with different types.
- For example, the **+ operator** will, perform **arithmetic addition** on:
  - **two numbers**
  - **merge two lists** and
  - **concatenate two strings**.
- This feature in Python, that allows same operator to have different meaning according to the context is called **operator overloading**.

What happens if we use "+" on objects of a user defined class?

Operator Overloading

Can we use "+" to add two points in a Cartesian system?

```
class Point:
        def __init__(self,x = 0,y = 0):
                self.x = x
                self.y = y
```

```
>>> p1 = Point(2,3)
>>> p2 = Point(-1,2)

>>> p1 + p2

Traceback (most recent call last): ... TypeError: unsupported
operand type(s) for +: 'Point' and 'Point'
```

- **Python didn't know how to add two Point objects together.**
- We can teach this to Python through operator overloading, using special functions.

Overloading + Operator

To overload the + sign, we will need to implement __add__() function in the class.

```
class Point:
# previous definitions...
  def __add__(self,other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x,y)
  def __init__(self,x = 0,y = 0):
        self.x = x self.y = y
  def __str__(self):
         return "({0},{1})".
          format(self.x,self.y)
```

```
>>> p1 = Point(2,3)
>>> p2 = Point(-1,2)
>>> print(p1 + p2)

Output:
(1,5)
```

- What actually happens is that, when you do p1 + p2, Python will call p1.__add__(p2) which in turn is Point.__add__(p1,p2).
- Similarly, we can overload other operators as well.

Operator Overloading Special Functions in Python

Operator	Expression	Internally
Addition	p1 + p2	p1.__add__(p2)
Subtraction	p1 - p2	p1.__sub__(p2)
Multiplication	p1 * p2	p1.__mul__(p2)
Power	p1 ** p2	p1.__pow__(p2)
Division	p1 / p2	p1.__truediv__(p2)
Floor Division	p1 // p2	p1.__floordiv__(p2)
Remainder (modulo)	p1 % p2	p1.__mod__(p2)
Bitwise Left Shift	p1 << p2	p1.__lshift__(p2)
Bitwise Right Shift	p1 >> p2	p1.__rshift__(p2)
Bitwise AND	p1 & p2	p1.__and__(p2)
Bitwise OR	p1 | p2	p1.__or__(p2)
Bitwise XOR	p1 ^ p2	p1.__xor__(p2)
Bitwise NOT	~p1	p1.__invert__()

Overloading Comparison Operators in Python

- We can overload comparison operators as well.
- Suppose, we wanted to implement the less than symbol < symbol in our Point class.
- Let us compare the magnitude of these points from the origin and return the result for this purpose. It can be implemented as follows.

```
class Point:
# previous definitions...
        def __lt__(self,other):
                self_mag = (self.x ** 2) + (self.y ** 2)
                other_mag = (other.x ** 2) + (other.y ** 2)
                return self_mag < other_mag
```

```
>>> Point(1,1) < Point(-2,-3)
True
>>> Point(1,1) < Point(0.5,-0.2)
False
>>> Point(1,1) < Point(1,1) False
```

Comparison Operator Overloading in Python

Operator	Expression	Internally
Less than	p1 < p2	p1.__lt__(p2)
Less than or equal to	p1 <= p2	p1.__le__(p2)
Equal to	p1 == p2	p1.__eq__(p2)
Not equal to	p1 != p2	p1.__ne__(p2)
Greater than	p1 > p2	p1.__gt__(p2)
Greater than or equal to	p1 >= p2	p1.__ge__(p2)

Examples & Case Study

Thank You